

Laua broneerimise veebirakendus

Marten Tiisler

Info:

Kasutajaliides on loodud [Vue.js](#) raamistikuga ning olekuhaldusraamistikuga Pinia. Front-end on majutatud Netlify teenuses.

Rakendus suhtleb Spring Boot backendiga, mis kasutab andmete haldamiseks mälusisest H2 andmebaasi. Backend on paigutatud Docker konteinerisse ning jookseb Render platvormil.

Kokku kulus ligikaudu 30-40 tundi.

Mis oli kerge ja mis oli raske:

- Kõige kergem oli kindlasti frontend disaini ja loogika arendus, sest nii [Vue.js](#) ning Pinia kasutamisega olid mul juba eelnevad kogemused, seega seal väga suuri raskusi ei esinenud. Kui aeg-ajalt ajas mõni asi segadusse, siis piisas tavaliselt dokumentatsiooni lugemisest või lihtsalt googeldamisest. Frontendi kõige raskemaks osaks arvasin olevat tõlkefunktsiooni loomist, kuid kui sain aru kuidas seda Pinia abil teha, osutuks see tegelt hoopis väga kergeks ja loogiliseks.
- Raskeimaks osutus Spring Bootiga tegelemine, sest sellega eelnevad kogemused puudusid ning seal nõutavad annotatsioonid tekitavad algajana suurt segadust. Jällegi proovisin eelkõige hakkama saada dokumentatsioonide ning õpetuste lugemise ja vaatamise kaudu. See-eest ei pidanud ma aga siin valeks seda, et küsida spetsiifilisemate ja raskemate kohtade juures tehisintellektilt natuke põhjalikumalt selgitust, et ikkagi kõigest paremini aru saada. Kui kasutasin pikemaid genereeritud koodijuppe, siis viitasin neile koodis ja siin dokumentatsioonis.

Näited tekkinud probleemidest:

- Tekkis probleem, et samal kellaajal oli iga laua broneeringu staatus täpselt sama. Koodi analüüsid avastasin, et "Random juhuslikkus = new Random(seed)" loomine toimus laudu läbiva tsükli sees, seega juhuslikkus.nextFloat() mitte ei võtnud iga laua jaoks eraldi suvalist arvu, vaid kasutati ainult seda algset juhuslikku arvu ja loodi see iga tsükliga uuesti.
- Kui lisasin sellise funktsiooni, et peale laua broneerimist muutuks laud broneerituks (nii frontendis kui backendis), siis hakkasid ilmneva erinevad probleemid. Näiteks oli alguses laud peale broneerimist broneeritud igavesti. Selle parandamise käigus hakkasid tekkima ka erinevad muud probleemid - näiteks oli vahepeal lehekülge värskendades laud täielikult puudu või oli neil kõigil staatus "broneerimata". Üritasin neid probleeme küll ise lahendada, kuid tundus, nagu iga parandus tõi esile mitu uut probleemi. Lõpuks kasutasin tehisintellekti abi nende probleemide likvideerimiseks. See muutis põhiliselt klassi LauaTeenuse meetodit saaLauadAjaPohjal. Tehtud muudatused tagasid, et laud saab kolmeks tunniks (rakendusesisese valitava aja mõistes) peale broneerimise nupu vajutamist staatuse broneeritud, ning selle aja

lõppedes genereeritakse aeg jällegi suvaliselt. Lisaks tegi AI paranduse, et suvalisuse seedi genereerimisel kasutaks rakendus valemit kuupäev + intervall (30min), mitte kuupäev + kellaeg. See lahendas ära laudade laadimise probleemid ning muutis broneeringute simuleerimise realistlikumaks.

- Backendi jaoks kasutatavas Renderi keskkonnas kasutan ma tasuta paketti. Tuli aga välja, et tasuta paketis uinub mu backend juba siis, kui sellele 15 minutit ühtegi päringut ei tule. See on ebamugav, sest kui nüüd keegi mu veebirakendust katsetada soovib, siis võtaks laudade ilmumine ehk backendi taaskäivitumine 1-2 minutit ja selle aja jooksul võib külastaja juba arvata, et rakendus on katki. Selle vältimiseks kasutan lehekülge cron-job.org, mis võimaldab mul automaatselt backendile saata iga 14 minuti tagant päringu, seega see ei uinu kunagi.

Funktsionaalsused:

- Kasutajale on kuvatud majaplaan, kus on selgelt eraldatud erinevad ruumid, seinad, mängunurk ning peenikeste siniste joontega on näha ka aknad.
- 30 minutiliste intervallide kaupa (rakendusesisene aeg) genereeritakse erinevad laudade seisud, kus igale lauale antakse suvaline broneeringu staatus. Broneeritud on umbes 40% laudadest.
- Samal kuupäeval ja kellaajal jääb laudade seis alati samaks. Selle jaoks tehakse seed (kuupäev + intervall). See antakse sisendiks Random objektile, mis omakorda otsustab iga laua broneeringu.
- Kasutaja saab valida kuupäeva, kellaaja, külaliste arvu, ruumi (tavaline saal, privaatne ruum, veranda) ning eelistused (mängunurga lähedal, vaikne nurk, akna all). Nende eelistuste põhjal soovitatakse talle sobivaim laud ning märgitakse ära, millised on tema jaoks selle laua plussid ja miinused.
- Kasutaja saab soovitud laua asemel muidugi vajutada ka teiste laudade peale. Ka nende puhul näidatakse talle, mis on tema eelistuste mõistes selle laua plussid ja miinused.
- Soovituse algoritm soovitab ainult selle ruumi laudu, mida kasutaja soovis. Lisaks ei soovitata muidugi laudu, mis mahutavad vähem inimesi kui külalisi on märgitud. Iga sobiva laua korral annab iga kattuv eelistus +10 punkti skoorile. Soovitatavaks lauaks tagastatakse laud, mille skoor on kõige kõrgem. Kui mitmel laual on võrdne skoor, siis soovitatakse seda, mille kohtade arv on kõige lähemal külaliste arvule.
- Kui kasutaja broneerib laua, siis on see rakendusesisese aja järgi 3 tundi broneeritud. Seega kui broneerida laud kell 14.00, siis 3 järgnevat tundi on laud broneeritud, kuid valides kellaks näiteks 17.30 võib laud olla juba uuesti vaba.
- Kasutaja saab ekraani paremal üleval nurgas valida eesti keele ja inglise keele vahel. Selle funktsiooni jaoks on tehtud Pinia store nimega keelteStore.js. See jätab meelde mis keel kasutajal valitud on ning oskab selle põhjal talle iga teksti kohta anda kas selle ingliskeelse või eestikeelse variandi.
- Veebirakendus toimib ka väiksemate ekraanide peal. Sealhulgas muutub väiksemate ekraanide peal ka menüüde paigutus, et laua broneerimine oleks mugav ka telefonis.
- Projekti frontend on hostitud Netlifys ja Dockeri kesta pandud backend jookseb Renderi keskkonnas.

Tähtsamad failid:

Front-end:

Front-endis on "src/components" kaustas kõik komponendid ehk paneelid, mis rakenduses kuvatakse. Seega on seal saali plaan (laudadePaneel.vue), kasutaja eelistuste paneel (kasutajaEelistused.vue), valitud laua paneel (valitudLauaPaneel.vue) ning broneeringu kinnitamise paneel (BroneeringuPaneel.vue). Nende failide sees toimub peamine vastavate paneelide sisu ja disaini loomine.

Need kõik kasutavad ka Pinia olekuhaldust, mille vastavad failid asuvad "src/stores" kaustas. "[laudadeStore.js](#)" on laudadega seotud info hoidmiseks ning backendiga läbi API suhtlemiseks. "[keelteStore.js](#)" aga on vajalik tõlkefunktsiooni täitmiseks ning hoiab endas muutujatena kõiki tekste, mis rakenduses kasutatakse.

"App.vue" on fail, kus on välja kutsutud kõik eelnevalt nimetatud komponendid ning kus toimub üldisem stiili ja paigutuse loomine.

Backend:

"controller" kaustas asub LauaController.java, mis käsitleb kõiki HTTP päringuid front-endist - laudade saamine kuupäeva/kellaaja alusel, parimad soovitud kasutaja eelistuste põhjal ning reserveeringud.

"model" kaustas asuvad andmeklassid: RestoraniLaud.java esindab andmebaasis salvestatavat lauda selle parameetritega, BroneeringuAndmed.java kannab broneeringu infot ning KasutajaEelistused.java kogub kasutaja eelistused.

"service" kaustas asub LauaTeenus.java, mis sisaldab kogu ärilooikat - laudade filtreerimist aja alusel, parima laua otsimist kasutaja eelistuste põhjal ning reserveeringu loomist. See klass ühendab kontrolleri ja andmebaasi.

"repository" kaustas on LauaRepository.java, mis käsitleb andmebaasisuhtlust - laudade salvestamist, laadimist ja otsimist. See kasutab Spring Data JPA-t, mis loob automaatselt SQL päringud.

BackendTableReservationApplication.java on backendi peafail ja stardipunkt.